



UNIVERSIDAD TÉCNICA DE MANABÍ

MATERIA:

DESARROLLO DE SISTEMAS INFORMATICOS

APELLIDOS Y NOMBRES:

Jover Argenis Holguin Lino

CORREO ELECTRÓNICO:

jholguin4662@utm.edu.ec

PROFESOR:

Ing. Jorge Arun Zambrano Cedeño, Mg.

Docente | Desarrollo de Sistemas Informáticos

TEMA:

Actividad #1:

Estudio de Caso - Sistema de Gestión de Incidentes (Help Desk)

SEMESTRE:

5°

PARALELO:

“A”

PERIODO ACADÉMICO:

Abril 2026 - Agosto 2026



Introducción

Actualmente las empresas necesitan sistemas eficientes para gestionar incidentes técnicos y solicitudes de soporte. Un sistema Help Desk permite registrar, organizar y dar seguimiento a problemas reportados por los usuarios, mejorando la atención y optimizando el trabajo del personal técnico.

En este estudio de caso se realizará el diseño lógico de un sistema de gestión de incidentes utilizando herramientas de modelado UML, patrones de diseño y metodologías profesionales de control de versiones mediante GitHub y Gitflow.

Análisis del Sistema

El sistema Help Desk permitirá registrar incidentes técnicos reportados por los usuarios de una organización. Los tickets podrán ser asignados a técnicos especializados para su resolución y seguimiento.

El sistema tendrá control de estados, prioridades, categorías y notificaciones automáticas para mantener informado al usuario sobre el progreso del incidente.

Además, el administrador podrá supervisar el rendimiento del soporte técnico mediante reportes y estadísticas.

3. Actores del Sistema

El sistema de gestión de incidentes Help Desk tiene como objetivo facilitar el registro, control y seguimiento de problemas técnicos reportados por los usuarios dentro de una organización.

Actualmente muchas empresas presentan dificultades en la administración de incidencias debido al uso de procesos manuales, falta de seguimiento y desorganización en la asignación de tareas técnicas. Esto provoca retrasos en la solución de problemas y una baja eficiencia en el soporte técnico.

El sistema Help Desk permitirá que los usuarios registren incidentes mediante tickets, especificando información como la categoría del problema, descripción, prioridad y estado del



incidente. Una vez registrado el ticket, el sistema asignará el caso a un técnico responsable para su atención y seguimiento.

Los técnicos podrán actualizar el estado del incidente, agregar observaciones y registrar las soluciones aplicadas. Además, el administrador tendrá acceso a reportes y estadísticas para supervisar el rendimiento del área de soporte técnico.

El sistema busca mejorar la organización del soporte, optimizar los tiempos de respuesta y mantener un control eficiente de todos los incidentes reportados.

4. Requerimientos Funcionales

El sistema Help Desk deberá cumplir con los siguientes requerimientos funcionales:

1. El sistema deberá permitir el registro de usuarios mediante credenciales de acceso.
2. El sistema deberá permitir el inicio y cierre de sesión de usuarios autorizados.
3. El usuario podrá registrar incidentes técnicos mediante la creación de tickets.
4. El sistema deberá permitir clasificar los incidentes según su categoría y nivel de prioridad.
5. El administrador podrá asignar tickets a técnicos especializados.
6. Los técnicos podrán actualizar el estado de los incidentes (pendiente, en proceso, resuelto o cerrado).
7. El sistema deberá almacenar el historial de cambios y observaciones realizadas en cada ticket.
8. Los usuarios podrán consultar el estado actual de sus incidentes.
9. El sistema enviará notificaciones sobre cambios o actualizaciones en los tickets registrados.
10. El administrador podrá generar reportes y estadísticas sobre incidentes y rendimiento del soporte técnico.
11. El sistema permitirá administrar usuarios, técnicos y roles de acceso.
12. El sistema deberá mantener la información almacenada de manera segura y organizada.

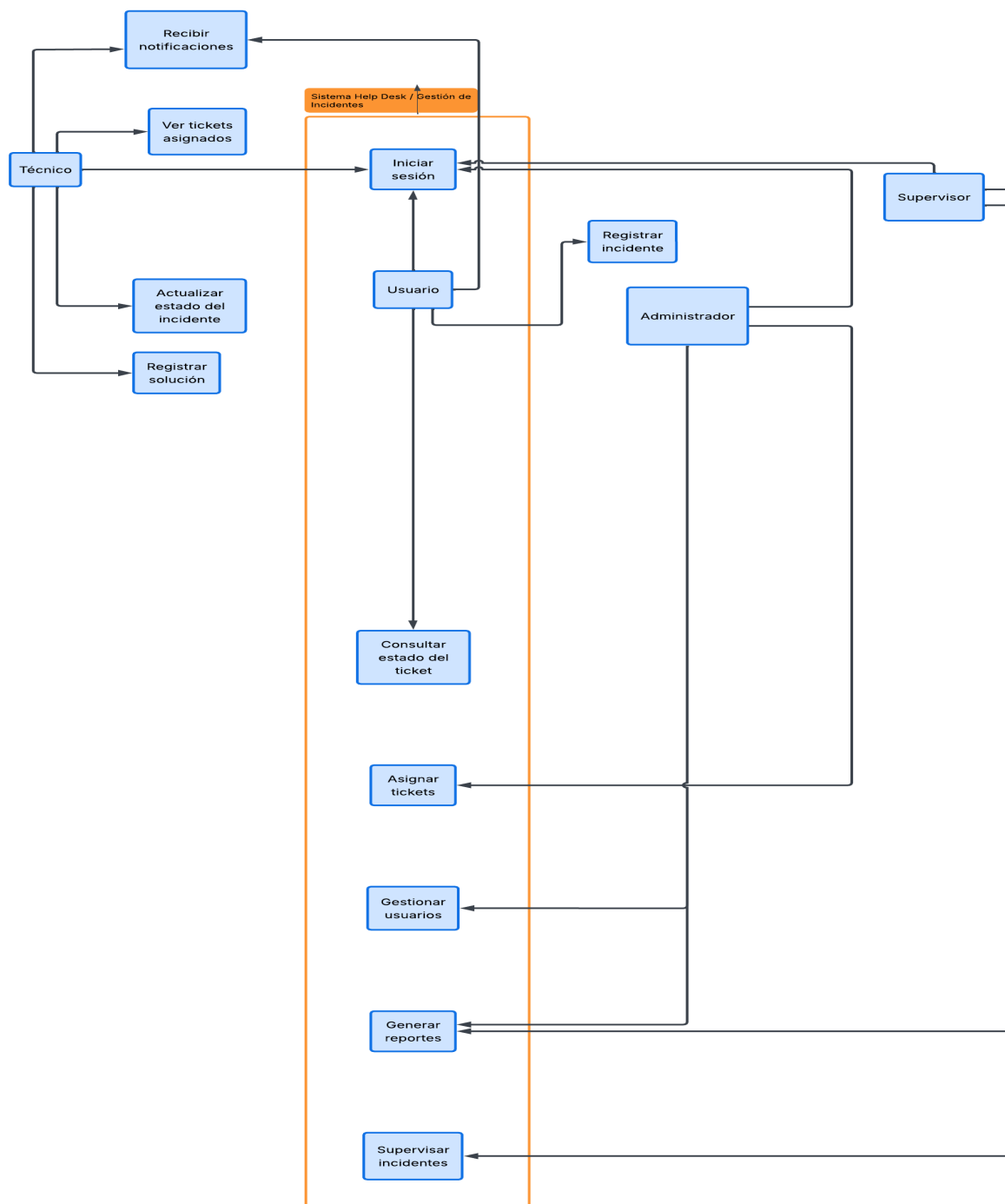


5. Diagramas UML

En esta sección se presentan los diagramas UML desarrollados para el sistema Help Desk / Gestión de Incidentes. Estos diagramas permiten representar gráficamente la estructura y funcionamiento del sistema, facilitando la comprensión de los procesos, actores y relaciones entre los diferentes componentes del software.

5.1 Diagrama de Casos de Uso

El diagrama de casos de uso representa las interacciones entre los actores del sistema y las funcionalidades principales del Help Desk. Los actores identificados son Usuario, Técnico, Administrador y Supervisor, quienes interactúan con diferentes procesos como registrar incidentes, asignar tickets, actualizar estados y generar reportes.

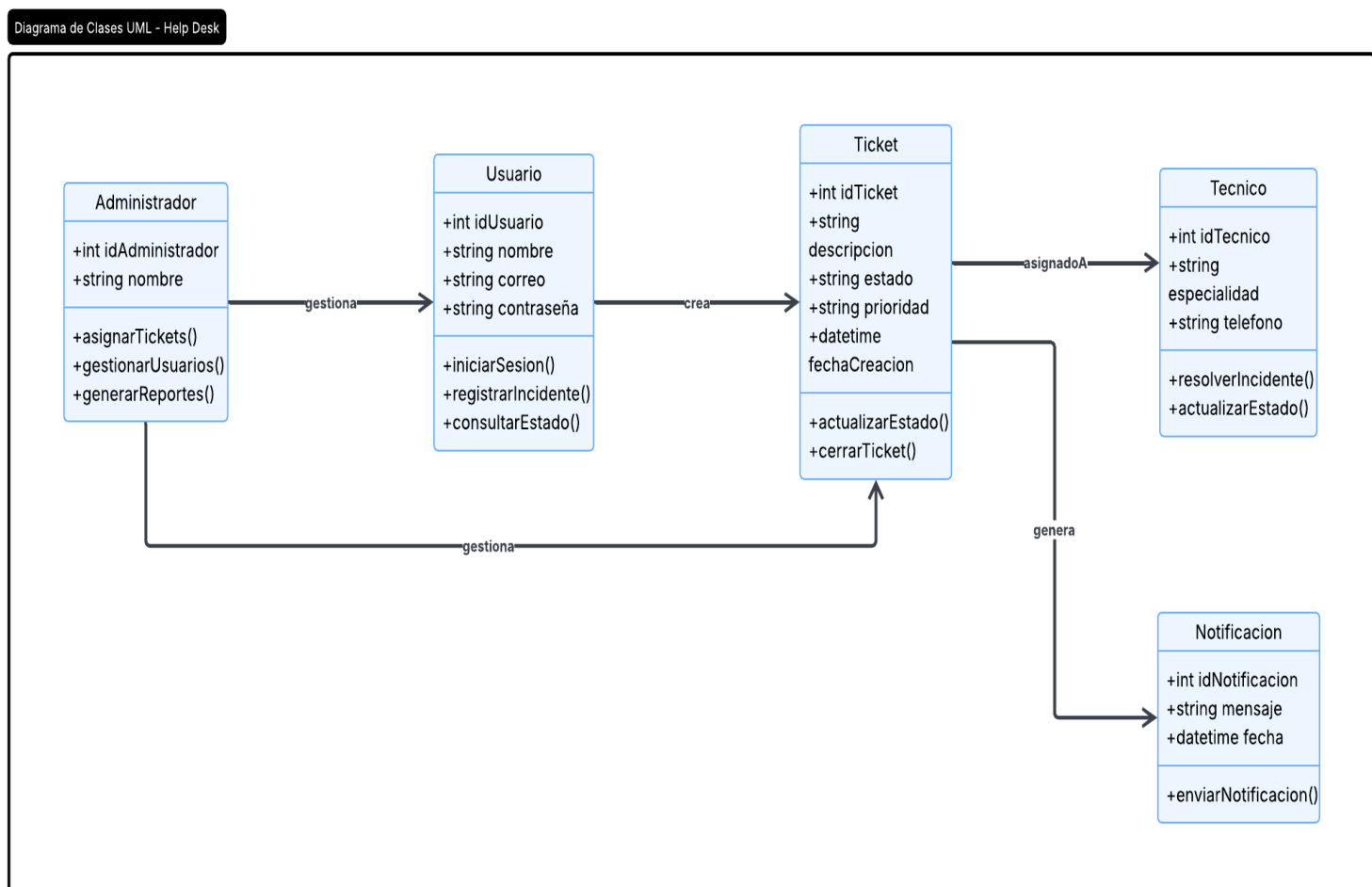




https://lucid.app/lucidchart/061c7f83-bf29-420e-afd0-0e363c8436e0/edit?viewport_loc=1309%2C-2845%2C2876%2C1263%2C0_0&invitationId=inv_c73e699c-9616-43d4-b0cc-add746fa94bf

5.2 Diagrama de Clases

El diagrama de clases representa la estructura lógica del sistema, mostrando las principales clases, atributos y relaciones existentes entre los objetos del sistema Help Desk. Este diagrama permite visualizar cómo se organiza la arquitectura interna del software.



https://lucid.app/lucidchart/4e553aa6-5525-4c72-9b52-f1a2f4281685/edit?view_items=KZ2XVFR5snHP&page=0_0&invitationId=inv_776fc561-abfc-4064-9c16-d3f4fa360850



Para el desarrollo del sistema Help Desk se aplicarán distintos patrones de diseño con el objetivo de mejorar la organización, escalabilidad y mantenimiento del software.

6.1 Patrón Singleton

El patrón Singleton se utilizará para garantizar que exista una única instancia de configuración o conexión principal del sistema. Esto permitirá controlar de manera eficiente el acceso a recursos compartidos, evitando duplicidad de conexiones y mejorando el rendimiento del sistema.

6.2 Patrón Factory Method

El patrón Factory Method será implementado para la creación de diferentes tipos de tickets o incidentes, como problemas de hardware, software o red. Este patrón permitirá crear objetos de manera flexible sin depender directamente de clases específicas.

6.3 Patrón Observer

El patrón Observer se utilizará para gestionar las notificaciones automáticas del sistema. Cada vez que un ticket cambie de estado, el usuario recibirá una notificación sobre la actualización del incidente, permitiendo mantener una comunicación constante entre el sistema y los usuarios.

La implementación de estos patrones de diseño permitirá construir un sistema más modular, reutilizable y fácil de mantener, siguiendo buenas prácticas de ingeniería de software.

7. Uso de Gitflow y GitHub

Para el desarrollo y organización del sistema Help Desk se utilizará GitHub como plataforma de control de versiones y almacenamiento del proyecto. Además, se implementará la metodología Gitflow para mantener un flujo de trabajo ordenado y profesional durante el desarrollo del sistema.

Gitflow permite dividir el trabajo en diferentes ramas, facilitando el desarrollo de nuevas funcionalidades, corrección de errores y control de versiones del proyecto.

Las ramas utilizadas serán las siguientes:



main: Contendrá la versión estable y final del proyecto.

develop: Será la rama principal de desarrollo donde se integrarán los avances del sistema.

feature/analisis-requerimientos: Rama destinada al análisis y documentación de requerimientos.

feature/diagramas-uml: Rama utilizada para el desarrollo de diagramas UML.

feature/patrones-diseno: Rama destinada a la implementación y documentación de patrones de diseño.

feature/documentacion-final: Rama utilizada para la elaboración del documento final del proyecto.

El uso de GitHub permitirá registrar cambios, controlar versiones, mantener respaldo del proyecto y facilitar el trabajo colaborativo entre los miembros del equipo de desarrollo.

La aplicación de Gitflow mejora la organización del proyecto y permite seguir buenas prácticas utilizadas en entornos profesionales de ingeniería de software.

8. Conclusiones

El desarrollo del presente estudio de caso permitió comprender la importancia del análisis y diseño arquitectónico en la construcción de sistemas de software profesionales. Mediante la aplicación de diagramas UML fue posible representar de manera clara la estructura y funcionamiento del sistema Help Desk.

La implementación de patrones de diseño como Singleton, Factory Method y Observer permitió identificar soluciones reutilizables para mejorar la organización, escalabilidad y mantenimiento del sistema. Asimismo, el uso de GitHub y la metodología Gitflow ayudó a comprender la importancia del control de versiones y la correcta administración del flujo de trabajo durante el desarrollo de software.

Finalmente, esta actividad fortaleció las capacidades de análisis, modelado y planificación de soluciones tecnológicas, aplicando buenas prácticas de ingeniería de software orientadas al desarrollo de sistemas eficientes y mantenibles.



9. Referencias

- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
- Pressman, R. S. (2015). Ingeniería del Software: Un Enfoque Práctico. McGraw-Hill Education.
- Sommerville, I. (2016). Ingeniería de Software. Pearson Educación.
- OMG Object Management Group. (2023). Unified Modeling Language (UML). Recuperado de <https://www.uml.org/>
- GitHub. (2024). GitHub Documentation. Recuperado de <https://docs.github.com/>
- Atlassian. (2024). Gitflow Workflow. Recuperado de <https://www.atlassian.com/git/tutorials/comparing-workflows/gitflow-workflow>